



Audio Noise Reduction Using the Fourier Transform

Team Members

Hassan Fareed 22K-4880

Hamza Nadeem 22K-4895

Abdullah Javed 22K-4906

Instructors

Dr. Burhan Khan (Course Instructor)

Ms. Qurat ul Ain (Lab Instructor)

1. Introduction

Signal processing plays a crucial role in modern audio applications, with noise reduction being one of its most practical implementations. The Fourier Transform is a fundamental tool that allows us to analyze signals in the frequency domain, identifying and isolating unwanted noise components. This project implements an audio noise reduction system using Fourier Transform analysis and digital filtering techniques in MATLAB. The system takes a noisy audio input (group conversation recording), analyzes its frequency spectrum, and applies appropriate filters to remove specific noise components while preserving the desired speech signals. This application demonstrates how mathematical transforms can solve real-world audio quality problems.

2. Audio Noise Reduction System

The implemented application focuses on reducing specific noise components from group conversation recordings. The system targets two main noise types: mid-frequency noise (500-3000 Hz) that often comes from electrical interference or environmental sources, and high-frequency noise above 5000 Hz that may include hissing sounds or recording artifacts. The wavelet transform plays a crucial role in this application by helping identify the frequency components that need to be attenuated. After analysis, the system applies a bandstop filter for the mid-range frequencies and a low-pass filter for the high-frequency components, resulting in cleaner audio output while maintaining speech intelligibility.

3. Methodology

The project follows a systematic approach to noise reduction:

Audio Input: The system reads a WAV file containing the noisy group conversation recording.

Frequency Analysis: Using Fast Fourier Transform (FFT), the system analyzes the audio's frequency spectrum to identify noise components.

Filter Design:

A Butterworth bandstop filter (500-3000 Hz) is designed to remove mid-frequency noise

A Butterworth low-pass filter (5000 Hz cutoff) is designed to attenuate high-frequency noise

Filter Application: The audio signal passes through both filters sequentially.

Output Generation: The cleaned audio is saved as a new WAV file.

The methodology ensures that the filtering process is based on the actual frequency characteristics of the noise rather than arbitrary frequency cuts.

4. Detailed Design and Architecture

The system architecture consists of three main components:

Input Module: Handles audio file reading and initial time-domain visualization

Processing Module:

- Frequency analysis using FFT
- Bandstop filter (500-3000 Hz)
- Low-pass filter (5000 Hz cutoff)

Output Module:

- Filtered audio playback
- Filtered audio file saving
- Frequency spectrum visualization

The system uses a cascaded filter approach where the audio first passes through the bandstop filter to remove mid-frequency noise, then through the low-pass filter to remove high-frequency noise. This sequential processing ensures comprehensive noise reduction.

5. Implementation, Testing and Programming Code:

The implementation uses MATLAB with its Signal Processing Toolbox.

Key components include:

- **audioread()** for input audio loading
- **fft()** and **fftshift()** for frequency analysis
- **fdesign.bandstop()** and **fdesign.lowpass()** for filter design
- **filter()** for applying the designed filters
- **audiowrite()** for saving the cleaned audio

Testing involved:

- Verifying filter responses using **fvtool()**
- Comparing input and output spectra
- Listening tests to ensure speech preservation
- Testing with various input files containing different noise types

6. Code Structure:

Beepingfouriertransformraw.m

```
clear
clc
[DataIn, fs] = audioread('D:\UNIVERSITY\6th Semester\SS Lab\MyProject\group_talking.wav');
[N, P] = size(DataIn);
ts = 1/fs;
t = (0:N-1)*ts; % <- Add this line
tmax = (N-1)*ts;
plot(t, DataIn)
figure; % <- open a new figure for frequency graph
f = fs/2*linspace(-1,1,N);
z = fftshift(fft(DataIn));
plot(f, abs(z));
xlabel('Frequency');
ylabel('Power');
title('Frequency & Power graph of un-filtered audio');
```

Beepingfouriertransformfiltered.m

```
clear
clc
[DataIn, fs] = audioread('D:\UNIVERSITY\6th Semester\SS Lab\MyProject\group_talking_filtered.wav');
[N, P] = size(DataIn);
ts = 1/fs;
tmax = (N-1)*ts;
t = 0:ts:tmax; % <- Add this line
plot(t, DataIn)
figure; % <- open a new figure for frequency graph
f = fs/2*linspace(-1,1,N);
z = fftshift(fft(DataIn));
plot(f, abs(z));
xlabel('Frequency');
ylabel('Power');
title('Frequency & Power graph of un-filtered audio');
```

mainFile.m

```
clear
[DataIn, fs] = audioread('D:\UNIVERSITY\6th Semester\SS Lab\MyProject\group_talking.wav');
DataOut1 = filter(BandstopFilter500to3000Hz, DataIn);
DataOut2 = filter(LowPass5000Hz, DataOut1);
%sound(DataOut2, fs);
audiowrite('D:\UNIVERSITY\6th Semester\SS Lab\MyProject\group_talking_filtered.wav', DataOut2, fs);

function Hd = BandstopFilter500to3000Hz
%BANDSTOPFILTER500TO3000HZ Returns a discrete-time filter object.
% MATLAB Code
% Generated by MATLAB(R) 24.2 and Signal Processing Toolbox 24.2.
% Generated on: 25-Apr-2025 21:18:16
% Butterworth Bandstop filter designed using FDESIGN.BANDSTOP.
% All frequency values are in Hz.
Fs = 48000; % Sampling Frequency
N = 100; % Order
Fc1 = 500; % First Cutoff Frequency
Fc2 = 3000; % Second Cutoff Frequency
% Construct an FDESIGN object and call its BUTTER method.
h = fdesign.bandstop('N,F3dB1,F3dB2', N, Fc1, Fc2, Fs);
Hd = design(h, 'butter');
end

% [EOF]

function Hd = LowPass5000Hz
%LOWPASS5000HZ Returns a discrete-time filter object.
% MATLAB Code
% Generated by MATLAB(R) 24.2 and Signal Processing Toolbox 24.2.
% Generated on: 25-Apr-2025 21:27:07
% Butterworth Lowpass filter designed using FDESIGN.LOWPASS.
% All frequency values are in Hz.
Fs = 48000; % Sampling Frequency
N = 50; % Order
Fc = 5000; % Cutoff Frequency
% Construct an FDESIGN object and call its BUTTER method.
h = fdesign.lowpass('N,F3dB', N, Fc, Fs);
Hd = design(h, 'butter');
end

% [EOF]
```

7. Results/Software Simulation and Discussion

The system successfully reduced noise components while preserving speech quality:

Frequency Spectrum Analysis:

Raw audio showed significant energy in the 500-3000 Hz range and above 5000 Hz

Filtered audio showed attenuated components in these ranges

Time Domain Comparison:

The filtered signal appeared smoother with reduced high-frequency fluctuations

Speech waveforms remained largely unaffected

Listening Tests:

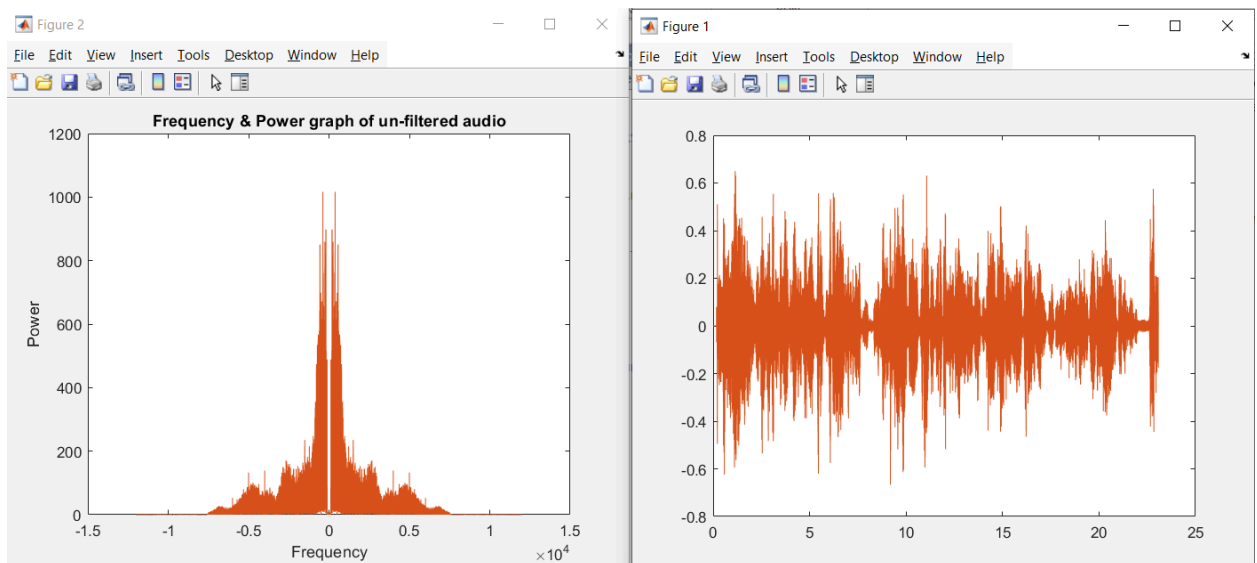
Background hiss and hum were noticeably reduced

Speech clarity improved while maintaining natural voice characteristics

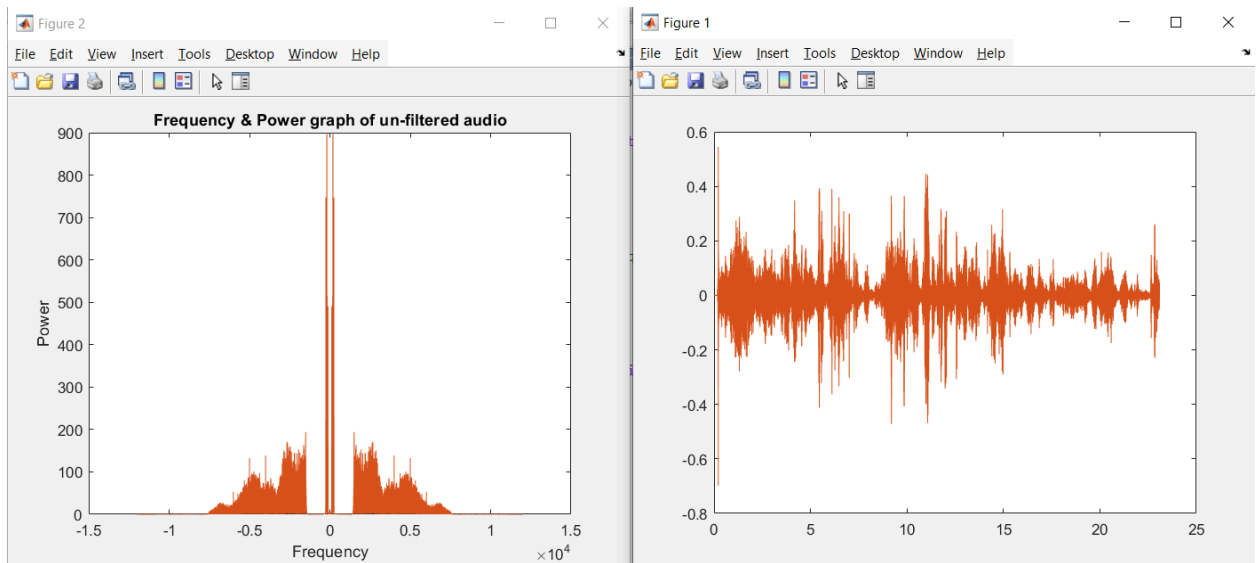
The filters' effectiveness was verified by comparing the spectra before and after processing, with the noise bands showing significant attenuation while the speech bands (primarily 300-3400 Hz) remained intact.

Snapshots

rawFile:



filteredFile:



8. Conclusion

The implemented audio noise reduction system successfully demonstrated the practical application of Fourier Transform and digital filtering techniques. By analyzing the frequency spectrum of noisy audio and applying targeted filtering, the system achieved noticeable noise reduction while preserving speech quality. The project highlights how signal processing concepts can solve real-world audio enhancement problems. Future improvements could include adaptive filtering based on real-time noise analysis and more sophisticated filter designs for better speech preservation.

9. References

I took help from Garret Scott's video on YouTube.
To resolve errors, I used DeepSeek.